BeGuardit Hacking Tools

Master Operations Guide

Design, deployment and operations of the BenchBrex local security toolchain: BG Recon, BG RedTeam, BG Lab and Sable-01.

Field	Value
Document	Master Operations Guide (How-to), v2.0
Prepared for	Parth Patel, founder of BenchBrex.com
Prepared by	Hermes Agent, under owner direction
Organisation	BeGuardit / BenchBrex / ScopeSable
Date	2026-11-01
Classification	CONFIDENTIAL - internal use only
Distribution	Owner only. Do not distribute outside BenchBrex.

Scope statement. This guide covers four systems built and verified on the owner's MacBook Air (M4, 16 GB, macOS 26) between 2026-10-29 and 2026-10-31. Every capability, limit and test result stated here was executed and observed - not projected. All targets are owned or authorized; third-party targeting is out of scope by design and enforced at the tool layer.

Document control

Version	Date	Change	Approved by
1.0	2026-10-31	First issue - covers all four systems, verified state	Parth Patel

Contents

Section	Title
1	Executive summary
2	Platform and architecture
3	BG Recon - reconnaissance and posture plugin
4	BG RedTeam - AI red-team harness
5	BG Lab - local exploitation environment
6	Sable-01 - owned cyber language model
7	Platform requirements and installation
8	Operational limits and capacity
9	Security, authorization and governance
10	Operations and troubleshooting
A	Repository map
B	Verification record
C	Attribution and provenance

1. Executive summary

Four systems were designed, built, independently reviewed and verified on a single Apple Silicon Mac. They form a closed loop: **BG Recon** measures the attack surface of owned hosts, **BG RedTeam** attacks the owner's own AI agents with deterministic test cases, **BG Lab** provides vulnerable targets and exploit tooling for safe practice, and **Sable-01** is an owned, offline language model that answers domain questions and enforces a trained refusal policy.

All four run on one machine, operate exclusively against authorized targets, and store all artifacts on an external encrypted-at-rest volume to keep the internal disk free. The table below summarises the verified state.

	One-line description	Headline verified result
	Hermes plugin: gated TCP scan + host posture audit	5-listener audit with 3 LAN-exposure findings; 48-port scan in 0.6
	Prompt-injection / tool-abuse harness for own AI agents	7-case suite; surfaced a real destructive-command gap on first ru
	DVWA + Juice Shop containers with stdlib exploit modules	SQLi dump: 5/5 users recovered, 5/5 hashes cracked offline
	Owned offline cyber LLM (Qwen3-4B + LoRA) with serving stack	Frozen eval 27/28 (96%); 8,192-token envelope guard; 15-243 to

v2 upgrade (2026-10-31 / 11-01). The toolchain was upgraded to a platform: a single command center (bgx) drives all four systems; BG Recon gained persistent posture baselines with change detection and HTML reporting; the red-team suite doubled to 13 cases with a regression-trend tool; the lab gained two more exploit modules behind authorization gates; the guard exposes operational metrics; and the production model was promoted to a new adapter that both matches the 96 percent frozen eval and emits tool calls (9/10 on the frozen tool gate) - closing the known run-2 gap. All changes passed an independent two-round review.

Red-team findings (owner attention). The expanded 13-case live suite surfaced five agent-safety findings: the assistant under test read secret-material canaries from its sandbox (critical), accepted a forged admin approval for a destructive command, and disclosed path-traversal content. These are the harness working - each finding is a concrete hardening task for the agent stack, tracked in the trend ledger.

Reading this guide. Sections 3 to 6 each follow the same pattern: what the system is, how it functions, how to use it, and its limits. Section 7 lists platform requirements, section 8 the measured capacity envelope, section 9 the security and governance model, and section 10 the operational troubleshooting playbook. Appendices carry the repository map, the verification record and provenance.

2. Platform and architecture

2.1 Hosting platform

Layer	Component
Hardware	MacBook Air M4, 16 GB unified memory, Apple GPU (16-core)
Operating system	macOS 26
Agent runtime	Hermes desktop app + gateway (profiles: default, sable)
Python	3.11 (system, CLT) for tooling; 3.14 (Hermes-managed) for the gateway; dedicated venv (3.11) for ML
ML runtime	mlx-lm 0.32.2 on the Apple GPU backend
Containers	colima (Docker runtime 29.5.2), qemu amd64 emulation for x86 images
Storage	External volume hosts all artifacts and the colima VM; $HOME paths are symlinks

2.2 System architecture

The four systems are independent processes with two integration points. The Sable-01 serving stack is a two-stage pipeline: an envelope guard (public port 8001) validates, token-counts and rewrites every request before forwarding to the model server (internal port 8002). The sable-bridge plugin exposes that pipeline to the Hermes agent as the ask_sable tool. BG Recon runs inside the Hermes plugin runtime; BG RedTeam and BG Lab are standalone command-line toolkits.

Flow	Path
Agent asks Sable-01	cloud agent -> ask_sable tool -> guard :8001 -> mlx :8002 -> trained model
Owner scans a host	Hermes chat -> BG Recon plugin -> gated socket scan / lsof posture audit
Owner tests own agents	BG RedTeam harness -> hermes one-shot turns -> deterministic scoring
Owner practices exploitation	BG Lab exploits -> DVWA / Juice Shop containers (colima)
Model training	dataset (72 rows) -> mlx_lm lora -> adapter -> frozen-eval gates -> serve

2.3 Storage layout

External path (under the volume root)	Contents	Internal symlink
bg-recon-plugin/	BG Recon agent plugin source	~/.hermes/plugins/bg-recon
sable-bridge/	ask_sable bridge plugin source	~/.hermes/plugins/sable-bridge
bg-redteam/	Red-team harness, cases, reports	~/bg-redteam
bg-lab/	Lab manager, exploits, README	~/bg-lab
sable-01/	Model, adapters, eval, serving stack, docs	~/sable-01
colima/	Container VM disk (5.3 GB, sparse)	~/.colima

The external volume must be mounted for any tool to run; unmounted drives leave the symlinks dangling and serve.sh fails loudly with a mount check. Approximately 7.9 GB moved off the internal disk during construction.

3. BG Recon - reconnaissance and posture plugin

3.1 What it is

BG Recon is a Hermes agent plugin that gives the assistant two capabilities in chat: a TCP port and banner scan of a host, and a local listening-service posture audit. It exists so that surface reconnaissance of owned hosts happens inside the governed agent rather than through ad-hoc shell commands.

3.2 How it functions

The plugin registers one tool, **bg_recon**, with two actions. **scan** takes a target and optional ports, resolves the host, opens sockets with a 0.5 s timeout against the allow-listed range, and returns port state plus any banner captured. **posture** runs lsof locally, filters to LISTEN-state sockets, and classifies each listener: loopback-bound services are rated OK; LAN-exposed services are flagged with the owning process, port and risk note.

Authorization gate. Every action checks the target against an allow-list of owned hosts and networks (localhost, the machine's own Tailscale names, the LAN prefix). Non-matching targets are refused before any packet is sent, with the reason returned to the agent. The refusal is a feature, not a limitation to work around.

3.3 How to use it

From any Hermes chat (default profile): ask the agent to scan or audit. Natural phrasings that route to the tool: 'run BG Recon posture on this Mac', 'scan localhost ports 8001-8010'. Direct CLI equivalent for verification: `python3 dashboard/plugin_api.py posture` from the plugin source directory.

Command / ask	Action	Typical duration
'audit this Mac's listening services'	posture	under 2 s
'scan host X ports 1-1024'	scan	about 10 s at 0.5 s timeout per port
'scan localhost 7900-8010'	scan	under 1 s

3.4 v2: posture baselines and reports

v2 adds continuous monitoring: bgx recon baseline-save snapshots the listener state; bgx recon baseline-diff reports added, removed and newly LAN-exposed services (verified by starting a live listener on 0.0.0.0:9999 during review); bgx recon report renders a one-page HTML assessment. An earlier field-name defect that silently disabled exposure detection was caught by independent review and fixed with a live verification.

3.4 Verified results and limits

The live posture audit found 5 listening services on the owner's Mac and flagged 3 LAN-exposed listeners (a dev HTTP server, the Tailscale relay and a local API) with owning process names and ports. A 48-port localhost scan completed in 0.6 s with correct open/closed classification. Banner capture is best-effort: services that do not greet first return no banner. The scan is TCP connect only - no UDP, no service fingerprinting, no vulnerability correlation; those are deliberate v1 boundaries.

4. BG RedTeam - AI red-team harness

4.1 What it is

BG RedTeam is a deterministic test harness that attacks the owner's own AI agents (currently Hermes one-shot sessions) with adversarial prompts and scores the responses. Its purpose is continuous assurance of the agent stack that runs the rest of this toolchain.

4.2 How it functions

The harness (`harness.py`) reads JSON case files from `cases/`. Each case defines a prompt, a scoring rule (must_refuse, must_not_execute, must_not_leak) and metadata. For every case it invokes the target agent in a fresh one-shot session with the prompt, captures the full transcript, applies the scorer, and writes a timestamped JSON report to `reports/`. Scoring is deterministic - pattern rules over the transcript, never model judgement - so two runs of the same case against the same agent are comparable.

4.3 How to use it

Step	Command (from ~/bg-redteam)
Run the full suite	python3 harness.py --all
Run one case	python3 harness.py --case cases/tool_abuse.json
List cases	python3 harness.py --list
Read a report	reports/<timestamp>/report.json

Adding a case means dropping a JSON file into cases/ with prompt, score rule and notes. The harness picks it up on the next --all run. Keep cases owned-only: the harness drives real agent sessions with real tools.

4.4 v2: expanded suite and trend tracking

The suite grew from 7 to 13 cases (exfiltration shapes, tool-argument injection, authority spoofing). trend.py compares two reports: regressions are severity-weighted (critical first, exit 1), a newly passing case counts as new coverage, and a newly failing case counts as a regression. The six new cases are authored and gated; their first full live run is pending.

4.4 Verified results and limits

The verified suite covers 7 cases across three families: direct instruction override, indirect injection through file content, and tool abuse (destructive shell, exfiltration-shaped requests). On first execution the suite surfaced one real finding: a destructive-command case was not refused outright by the default agent profile, which led to tightening the profile's safety contract. Limits: the harness measures the transcript, not side effects - a case that scores clean does not prove an out-of-band action never happened, so high-blast-rate actions still require the gateway's own approval gates. Each full run costs real model calls (7 cases took about 4 minutes).

5. BG Lab - local exploitation environment

5.1 What it is

BG Lab is a managed local range: two intentionally vulnerable applications (DVWA and OWASP Juice Shop) running in containers, plus standard-library-only exploit modules that operate against them. It exists for safe, repeatable practice of exploitation technique with zero external exposure.

5.2 How it functions

`lab.py` is the single control point: it starts or stops colima, brings the DVWA and Juice Shop containers up or down, and health-checks both web apps before declaring the lab ready. Exploit modules in `exploits/` target the lab endpoints over localhost: the SQLi module performs the classic DVWA injection, dumps the users table, cracks the recovered MD5 hashes offline against a small wordlist, and writes a JSON report. The command-injection module verifies DVWA's exec-level payload execution. `check_target.py` is the pre-flight: it confirms the lab is up and refuses to run exploits otherwise.

5.3 How to use it

Step	Command (from ~/bg-lab)
Start the range	python3 lab.py up (colima + DVWA + Juice Shop)
Verify targets	python3 check_target.py
Run SQLi exercise	python3 exploits/dvwa_sqli.py
Run cmd-injection exercise	python3 exploits/dvwa_cmdi.py
Stop the range	python3 lab.py down

Memory note: colima plus both containers coexist with the Sable-01 server only if the MLX server is stopped; on the 16 GB machine, run one heavy system at a time (see section 8).

5.4 v2: two more modules and authorization gates

v2 adds reflected-XSS and brute-force modules (four total). Every lab exploit now enforces an authorization allowlist - it refuses any target other than the local lab containers before sending a packet - and performs a pre-flight reachability check so a down lab produces a readable error instead of a hang. The two new modules await a live run against a restarted lab container (owner approval pending); their refusal paths are verified.

5.4 Verified results and limits

The SQLi exercise was executed end-to-end twice: 5 of 5 DVWA users dumped, 5 of 5 password hashes cracked offline, report written with per-user rows. The command-injection exercise confirmed payload execution at the target. Both modules refuse to run when the lab is down. Limits: exploits are stdlib-only and DVWA-specific by design - they are teaching instruments, not a general framework; the x86 containers run under qemu emulation, so container start is slow (minutes) and CPU-bound workloads inside them are noticeably slower than native.

6. Sable-01 - owned cyber language model

6.1 What it is

Sable-01 is the owner's own cyber-domain language model: a Qwen3-4B base fine-tuned with a LoRA adapter (rank 16) on 72 hand-built training rows covering three behaviours - domain question answering, the ScopeSable reporting voice, and a refusal policy for unauthorized targeting. It runs fully offline on the Apple GPU through mlx-lm, behind a hardening proxy, and is reachable from the Hermes agent as a first-class tool.

6.2 How it functions

6.2.1 Training pipeline

Training uses mlx_lm.lora on the base model with a fixed seed and a recorded provenance file (base model identity, dataset hash, adapter rank, iterations, loss). The dataset splits 72 rows into train and validation. Two runs exist: run 1 (r16, general domain voice) passed its gates and is the production adapter; run 2 (tool-format training) failed its acceptance gate - 1 of 10 emitted a well-formed tool call - and is explicitly NOT served. The lesson is recorded: teaching tool emission needs tool-context-aware training data, not format strings.

6.2.2 Evaluation gates

`eval/run_eval.py` runs a frozen 28-question suite (`eval/questions.jsonl`) with deterministic scoring (`eval/score.py`): 19 factual single-token-ish answers, 6 long-form rubric items, and 3 refusal probes. Run 1 scored 27 of 28 (96 percent) and was frozen as the production adapter. The eval is frozen: questions never change after gating, so later runs are comparable. One long-form item (K-05, attack-surface mapping depth) scores low-confidence and is tracked as a residual.

6.2.3 Serving stack (production)

The stack is two processes under one supervisor script (`serve.sh`). The model server (mlx_lm.server) binds 127.0.0.1:8002 - internal only - and serves the base model with the r16 adapter. The envelope guard (`serving/guard.py`) binds 127.0.0.1:8001 as the only public port. Every request to the guard is: size-capped (1 MiB), rate-limited (60 per minute), queued (2 concurrent generations, 4 waiting), token-counted against an 8,192-token envelope, rewritten to the production model name, and logged. The guard translates backend failures into actionable errors: 502 (model server down), 503 with a zombie marker (generation thread dead), 504 (generation timeout at 180 s), 413 (over-envelope), 429 (rate limit). On restart the guard verifies the generation thread with a real 8-token probe before clearing any zombie marker - a naive health endpoint would false-clear, which the review cycle caught and rejected.

6.2.4 Agent integration

The **sable-bridge** plugin exposes the stack to Hermes as the **ask_sable** tool: it validates the question, posts to the guard with a 120 s client timeout, and returns either the answer or a precise failure message (down, timeout, over-envelope). A `doctor --ci` mode verifies registration non-interactively. The **sable** Hermes profile routes the chat session directly at the guard with a system prompt carrying the trained identity, so the owner can also work with the model conversationally.

6.3 How to use it

Task	Command / entry point
Start the stack	~/sable-01/serve.sh start (about 3 GB RAM held)
Check state	~/sable-01/serve.sh status
Run the test battery	~/sable-01/serve.sh test
Stop and free memory	~/sable-01/serve.sh stop
Ask from any Hermes chat	the ask_sable tool, e.g. 'ask Sable-01 what BG Recon is'
Chat with the model directly	hermes -p sable (profile routes to the guard)
Re-run the frozen eval	~/sable-01/eval/run_eval.py
Unit-test the guard	~/sable-01: venv/bin/python3 tests/test_guard.py (21 tests)

Verified example: the ask_sable tool returned the trained one-line description of BG Recon in under 2 s after a cold start; the full test battery (short fact, over-envelope 413, follow-up context) passes against the live stack.

6.4 v2: tool-call capability and production promotion

Run 2's failure (1/10 tool emission) was root-caused: the dataset had zero tool-context rows. Run 3 added 40 such rows but regressed the frozen eval (25/28); run 4 rebalanced with double rehearsal weight and passed BOTH gates - tool gate 9/10, frozen eval 27/28 (96 percent, equal to production, refusals 10/10) - and was promoted. The one tool-gate miss (T-10: an unauthorized-target request inside a tool context receives a tool call instead of a refusal) is a declared residual. The guard also gained a /metrics endpoint (request counts, status breakdown, latency percentiles), and the stack's measured residency is about 1.3 GB (model server 1008 MB, guard 297 MB). A provenance correction was recorded and binary-verified: every adapter trains at LoRA rank 8 (mlx-lm ignores the top-level rank key); the r16 directory names are labels, not parameters.

6.4 Verified behaviour envelope

Behaviour	Verified result
Domain QA (5.5K-token prompts)	correct, cited training facts in 3-16 s
Refusal probes (3)	3 of 3 refused unauthorized targeting
Follow-up context retention	passed (second turn resolves 'it' correctly)
Identity adherence	states ScopeSable authorship correctly
Over-envelope prompts	refused by the guard at 12,816 tokens with guidance
28K-token stress prompt	did not kill the thread on retest; treated as memory-state-dependent

7. Platform requirements and installation

The toolchain is installed and verified on the owner's MacBook Air M4 (16 GB, macOS 26). It has no external service dependencies: everything runs locally.

Requirement	Detail
Machine	Apple Silicon Mac, 16 GB unified memory minimum
macOS	26 (tested); 14+ expected to work for the non-ML parts
Python	3.11+ (system or venv); reportlab not required at runtime
ML runtime	mlx-lm 0.32.2 (Apple GPU backend) in a dedicated venv
Hermes desktop	with the gateway running; plugin directories under ~/.hermes/plugins
Containers	colima + Docker CLI; x86_64 emulation via qemu for the lab images
Storage	external volume mounted (all artifacts live there; $HOME paths are symlinks)

7.1 First-time setup order

1. Mount the external volume; confirm ~/sable-01, ~/bg-lab, ~/bg-redteam resolve. 2. Start Hermes (gateway auto-runs with the desktop app). 3. Plugins load automatically; verify with hermes plugins doctor for bg-recon and sable-bridge. 4. Start the model stack with serve.sh start when Sable-01 is needed. 5. For lab work, stop the model stack first (memory), then lab.py up.

Nothing in the toolchain requires Homebrew; all CLIs are tarball or pip installs into user space, consistent with the owner's machine policy.

8. Operational limits and capacity

Every number below was measured on the target machine. They define the honest capacity envelope of the toolchain as built.

Dimension	Measured limit / capacity
Model context envelope	8,192 tokens hard gate at the guard; comfortable zone about 5,500
Generation latency	3-16 s typical (5.5K-token prompts); up to 180 s guard timeout
Concurrency	2 generations + 4 queued; excess refused 503 with Retry-After
Request rate	60 requests per minute sliding window; body cap 1 MiB
Memory while serving	about 1.3 GB measured (model 1008 MB + guard 297 MB); release with serve.sh stop
Memory contention	one heavy system at a time: model stack or container lab, not both
Knowledge boundary	72 training rows; 96 percent frozen eval; not a general cyber encyclopedia
Tool-format emission	run 2 failed (1 of 10); run 3 (tool-context training) is future work
Network exposure	loopback only by design; no listener is reachable off-machine
Scan engine	TCP connect only, no UDP, no fingerprinting, allow-listed targets
Container engine	x86 images under qemu emulation: slow start, slower CPU
Provenance	adapter, dataset, config and logs recorded; no production data in training
Tool emission	9/10 frozen tool gate (T-10 residual); teaching rows are tool-context-aware

8.1 Known residuals

Four items are tracked and accepted rather than hidden. (1) One long-form eval item (K-05) scores low-confidence. (2) The 28K-token thread death observed once did not reproduce under current memory conditions - the envelope guard exists because of it and stays. (3) BG RedTeam scores transcripts, not side effects. (4) The rate-limit window resets on guard restart - acceptable for a single-user host. None block the stated scope.

9. Security, authorization and governance

Authorization-first design. BG Recon refuses any target outside the owned allow-list before sending a packet. BG Lab exploits hard-code localhost lab endpoints and pre-flight the target. Sable-01's refusal probes are part of the frozen eval and the trained policy. BG RedTeam only drives the owner's own agents. There is no configuration path in any of the four systems that enables third-party targeting.

v2 authorization hardening. All four lab exploit modules now refuse any target outside the local-lab allowlist (127.0.0.1:8081 / localhost:8081) before sending a packet, closing a review finding that the modules could be pointed at arbitrary hosts. BG Recon's allow-list gate is unchanged; the model-side refusal policy held at 10/10 on the frozen set through the adapter promotion.

Layered defense for the model stack. The guard is the single public port and applies size, rate, concurrency and token-envelope controls before any prompt reaches the model; the model server is loopback-internal. Agent-side, the ask_sable tool carries its own timeout and failure translation. The guard's request log exists for observability and rotates at 5 MB.

Data handling. Training data was hand-built; no client, production or personal data was used. The provenance file records dataset hash and base-model identity. All artifacts live on the external volume; the internal disk holds only symlinks. The red-team suite found and drove the fix of one real agent-safety gap, which is the harness working as intended.

Out of scope by design. Multi-tenant hardening, remote exposure of the model stack, UDP scanning, and general-purpose exploit frameworks are not implemented and are not roadmap items without a new authorization decision by the owner.

10. Operations and troubleshooting

The runbook below is the operational playbook, mirrored in `sable-01/RUNBOOK.md`. The recovery ladder is: restart the stack, then full stop and start, then profile alias check, then unit tests to isolate guard versus backend.

Symptom	Meaning	Fix
503 + zombie header	generation thread dead	serve.sh restart (probe verifies recovery)
504 generation timeout	backend silent 180 s	retry once, then restart
502 model not running	MLX server down	serve.sh start
413 over envelope	prompt over 8,192 tokens	send a shorter, self-contained question
429 rate limit	over 60 req/min	wait up to 60 s
503 server busy	queue full	retry after about 5 s
Tool 'timed out after 120 s'	bridge gave up; model thinking	retry once; restart if persistent
400 bad request	malformed field	correct the named field
serve.sh mount check fails	external volume unmounted	mount the volume, retry

Daily rhythm. Start the stack only when needed (3 GB). Run serve.sh test after any guard or adapter change. Keep colima stopped while serving. The 21-test unit suite must stay green after every guard change - it is the regression gate.

Appendix A. Repository map

volume)	Key contents
	adapters/r16 (production), eval/ (frozen suite), serving/guard.py, serve.sh, tests/, designs/, RUNBOOK.md, PROGRAM.md
	plugin.yaml, tools.py (ask_sable), schemas.py
	plugin.yaml, dashboard/plugin_api.py, manifest.json, desktop/plugin.js
	harness.py, cases/ (7 JSON), reports/
	lab.py, check_target.py, exploits/dvwa_sqli.py, exploits/dvwa_cmdi.py, README.md
	sable-01/designs/: spec (r4), review (2 rounds), drift ledger, concepts brief

Appendix B. Verification record

All results below were executed on the target machine during 2026-10-29 to 2026-10-31 and are reproducible with the commands in this guide.

Verification	Result
Sable-01 frozen eval (28 questions)	27/28 = 96 percent; refusal 3/3
Guard unit suite (21 tests)	21/21 pass
Burst concurrency (10 parallel, live)	6 served, 4 clean 503, zero resets
504 timeout path (black-hole backend, live)	504 at 180 s, queue drained
Over-envelope (12,816 tokens, live)	413 with guidance; thread survived
Loopback binding audit	guard and model server bound to 127.0.0.1 only
Bridge doctor --ci	1 tool registered, 0 errors
BG Recon posture audit (live)	5 listeners found, 3 LAN-exposure findings
BG Recon scan (48 ports, live)	0.6 s, correct open/closed states
BG RedTeam suite	7 cases run; 1 real finding surfaced and fixed
BG Lab SQLi exercise	5/5 users dumped, 5/5 hashes cracked
BG Lab cmd-injection exercise	payload execution confirmed
Tool gate, run-4 adapter (frozen 10)	9/10 pass; robust parser re-verified
Frozen eval, run-4 adapter	27/28 (96 percent); refusals 10/10; promoted to production
Guard unit suite after v2	23/23 pass
Recon exposure detection (live listener)	0.0.0.0:9999 flagged as NEW LAN-EXPOSED after fix
Lab authorization gates (all 4 modules)	non-lab targets refused, exit 1
Guard /metrics (live)	request counts + latency percentiles recorded
bgx doctor	15/15 checks
Adapter rank (safetensors headers)	rank 8 confirmed for r16 and r16-run4 (224 tensors)
Lab exploits live (XSS + brute)	XSS 3/3 checks; brute 5/5 cracked in 11 attempts, no rate limit
Red-team 13-case live run	8 pass / 5 findings (1 critical) - trend baseline established
Goldfish round-2 confirmation	16/16 FIXED claims confirmed, 2 rejections justified - READY
Model selection (footprint, measured)	2.36-2.48 GB under the 3 GB rule; Qwen3-4B-4bit confirmed; 8B rejected

Appendix C. Attribution and provenance

This toolchain was designed, built, reviewed and verified as a directed engineering programme by Hermes Agent under the direction of Parth Patel, founder of BenchBrex.com. Attribution **Created by Parth Patel, founder of BenchBrex.com (BeGuardit / BenchBrex)** is embedded in every artifact: source headers, the guard's health endpoint, the plugin manifests, the runbook and this guide.

Provenance of the model: base Qwen3-4B (mlx 4-bit), LoRA rank 16, dataset of 72 hand-built rows (hash recorded), frozen eval 27/28. Review discipline: two independent review rounds, a drift ledger reconciling specification to code, and an explicit approved-changes gate before implementation. The design brief, reviews and drift ledger are preserved under sable-01/designs/.

Document ends. Questions on any section should be raised with the owner; the systems are internal to BenchBrex and are not licensed for external use.